Program 1 - Get the Largest Number from a List

```
list1=[]
while True :
    print("Enter an integer to the list:")
    element = int(input())
    list1.append(element)
    print("Is it end of list?(y/n)")
    flag = input()
    if flag == "y" :
        break
print("The List =", list1)
print("Maximum Value in this list:", max(list1))
```

Program 2 - Remove Duplicates from a List

```
list1=[20,20,10,30,30]
print("Duplicated List:", list1)
list2=[]
for i in list1:
    if i not in list2:
        list2.append(i)
print("Deduplicated List:", list2)
```

Program 3 - Conversion of Tuple to Dictionary

```
tuple = (('Rajesh', 75),('Vasudevan', 80), ('Thomas', 85))
print("Tuple=",tuple)
dictionary = dict(tuple)
print("Dictionary=",dictionary)
```

Program 4 - Merging Dictionaries

```
dict1 = {'a': 1, 'b': 2}
print("Dictionary 1=",dict1)
dict2 = {'c': 3, 'd': 4}
print("Dictionary 2=",dict2)
dict1.update(dict2)
print(dict1)
```

Program 5 - Checking Common Member in Lists

```
list1=[10,20,30]
set1=set(list1)
list2=[40,50,60]
set2=set(list2)
result=set1 & set2
if (result== set()):
    print("False")
else :
    print("True")
```

Program 6 - Find Earlier Date

```
date1="2025-01-26"
date2="2025-08-15"
if date1<date2 :
    print("Earlier date is", date1)
elif date1>date2 :
    print("Earlier date is", date2)
else:
    print("Two dates are equal")
```

Program 7 - Subtract 5 Days from Current Date

```
import datetime
today=datetime.date.today()
no_of_days=datetime.timedelta(days=5)
print(today-no_of_days)
```

Program 8 - File Copy Operation

```
source_file=open("file1.txt","r")
contents=source_file.read()
source_file.close()
destination_file=open("file2.txt","w")
destination_file.write(contents)
destination_file.close()
```

Program 9 - Capitalise Each Word in a File

```
file=open("file1.txt","r")
contents=file.read()
new_contents=contents.upper()
file.close()
file=open("file1.txt","w")
file.write(new_contents)
file.close()
```

Program 10 - Search and Replace in a File

```
file=open("file1.txt","r")
contents=file.read()
new_contents=contents.replace("Python","C")
file.close()
file=open("file1.txt","w")
file.write(new_contents)
file.close()
```

Program 11 - Count Number of Lines in a File

```
count = 0
file = open("file1.txt", "r")
for line in file:
    count = count + 1
print("Number of Lines:", count)
```

Program 12 - Retrieve Lines Having Two Consecutive 1's

```
lines_with_consecutive_ones = []
file = open("file1.txt", "r")
for line in file:
    if '11' in line:
        lines_with_consecutive_ones.append(line.strip())
print(lines_with_consecutive_ones)
print("Lines containing two consecutive '1's:")
for line in lines_with_consecutive_ones:
    print(line)
```

Program 13 - Matrix Operations Using Vectorisation

```
import numpy
matrix1=numpy.matrix([[1,2,3],[1,1,1],[1,1,1]])
matrix2=numpy.matrix([[1,0,0],[0,1,0],[0,0,1]])
print("Matrix 1=\n",matrix1)
print("Matrix 2=\n",matrix2)
matrix3 = numpy.add(matrix1,matrix2)
matrix4 = numpy.subtract(matrix1,matrix2)
matrix5 = numpy.matmul(matrix1,matrix2)
matrix6 = numpy.transpose(matrix1)
print("Matrix 1 + Matrix 2=\n", matrix3)
print("Matrix 1 - Matrix 2=\n", matrix4)
print("Matrix 1 * Matrix 2=\n", matrix5)
print("2 * Matrix 1=\n", 2*matrix1)
print("Transpose of Matrix 1=\n", matrix6)
```

Program 14 - Creation of Matrices for 2D Geometric Transformations

Translation Matrix

```
import numpy
def translationMatrix(tx=0, ty=0):
    return numpy.matrix([[1,0,tx],
                         [0,1,ty],
                         [0,0,1]])
matrix=translationMatrix(1,1)
print(matrix)
```

Rotation Matrix

```
import numpy
def rotationMatrix(degree):
    theta = numpy.radians(degree)
    c,s=numpy.cos(theta),numpy.sin(theta)
    return numpy.matrix([[c,-s,0],
                         [s, c,0],
                         [0,0,1]])
matrix=rotationMatrix(30)
print(matrix)
```

Scaling Matrix

```
import numpy
def scalingMatrix(sx=0, sy=0):
    return numpy.matrix([[sx,0,0],
                         [0,sy,0],
                         [0,0,1]])
matrix=scalingMatrix(2,2)
print(matrix)
```

Program 15 - Creation of Matrices for 3D Geometric Transformations

Translation Matrix

```
import numpy
def translationMatrix(tx=0, ty=0, tz=0):
    return numpy.matrix([[1,0,0,tx],
                         [0,1,0,ty],
                         [0,0,1,tz],
                         [0,0,0,1 ]])
matrix=translationMatrix(1,1,1)
print(matrix)
```

Rotation Matrix about x axis

```
import numpy
def rotationMatrix(degree):
    theta = numpy.radians(degree)
    c,s=numpy.cos(theta),numpy.sin(theta)
    return numpy.matrix([[1,0,0,0],
                         [0,c,-s,0],
                         [0,s,c,0],
                         [0,0,0,1]])
matrix=rotationMatrix(30)
print(matrix)
```

Rotation Matrix about y axis

```
import numpy
def rotationMatrix(degree):
    theta = numpy.radians(degree)
    c,s=numpy.cos(theta),numpy.sin(theta)
    return numpy.matrix([[c,0,s,0],
                         [0,1,0,0],
                         [-s,0,c,0],
                         [0,0,0,1]])
matrix=rotationMatrix(30)
print(matrix)
```

Rotation Matrix about z axis

```
import numpy
def rotationMatrix(degree):
    theta = numpy.radians(degree)
    c,s=numpy.cos(theta),numpy.sin(theta)
    return numpy.matrix([[c,-s,0,0],
                         [s,c,0,0],
                         [0,0,1,0],
                         [0,0,0,1]])
matrix=rotationMatrix(30)
print(matrix)
```

Scaling Matrix

```
import numpy
def scalingMatrix(sx=0, sy=0, sz=0):
    return numpy.matrix([[sx,0,0,0],
                         [0,sy,0,0],
                         [0,0,sz,0],
                         [0,0,0,1]])
matrix=scalingMatrix(2,2,2)
print(matrix)
```

Program 16 - Singular Value Decomposition of a Matrix

```
from numpy import matrix
from numpy import matmul
from numpy import diag
from scipy.linalg import svd
A = matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(A)
U, S, V = svd(A)
print(U)
print(S)
print(V)
Sigma = diag(S)
print(Sigma)
B = matmul(U,matmul(Sigma,V))
print(B)
```

Program 17 - Plot a Histogram

```
from matplotlib import pyplot
from numpy import array
fig,ax = pyplot.subplots(1,1)
a = array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
ax.hist(a, bins = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
ax.set_title("histogram of result")
ax.set_xticks([0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
ax.set_xlabel('marks')
ax.set_ylabel('no. of students')
pyplot.show()
```

Program 18 - Plot a Histogram Using Iris Data Set

```
from matplotlib import pyplot
import pandas
df = pandas.read_csv('iris.csv')
fig,ax = pyplot.subplots(1,1)
df['petal.length'].plot(kind='hist', edgecolor="black", bins=49)
ax.set_title("Histogram")
ax.set_xticks([1.0,1.5,2.0,2.5,3.0,3.5,4.0,4.5,5.0,5.5,6.0,6.5,7.0])
ax.set_yticks([0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19])
ax.set_xlabel('Petal Length')
pyplot.show()
```

Program 19 - Plot a Scatterplot

```
from matplotlib import pyplot
rollnos = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
marks = [22,87,5,43,56,73,55,54,11,20,51,5,79,31,27]
fig,ax = pyplot.subplots(1,1)
ax.scatter(rollnos, marks)
ax.set_title("Scatter Plot")
ax.set_xticks([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15])
ax.set_yticks([25,50,75,100])
ax.set_xlabel('rollnos')
ax.set_ylabel('marks')
pyplot.show()
```

Program 20 - Plot a Scatter plot Using Iris Data Set

```
from matplotlib import pyplot
import pandas
df = pandas.read_csv('iris.csv')
fig, ax = pyplot.subplots(1,1)
colors = {'Setosa':'red', 'Versicolor':'green', 'Virginica':'blue'}
grouped = df.groupby('species')
for key, group in grouped:
    group.plot(ax=ax, kind='scatter', x='petal.length', y='petal.width',
                label=key, color=colors[key])
ax.set_title("Scatter Plot")
ax.set_xlabel('Petal Length')
ax.set_ylabel('Petal Width')
pyplot.show()
```

Program 21 - Predict the Food Type Using k-NN algorithm

```
food=read.csv("food.csv")
tomato=data.frame(ingredient="tomato",sweetness=6,crunchiness=4)
food1=food[,2:3]
tomato1=tomato[,2:3]
k <- as.numeric(readline(prompt = "Enter the value of k:"))
cat("Enter the value of k:")
k=scan("stdin", integer(), n=1)
library(class)
pred=knn(food1,tomato1,food$FoodType,k)
cat("Predicted Food Type of Tomato:\n")
print(pred)
```

Program 22 - Classify a Student Using k-NN algorithm

```
library(class)
survey=read.csv("survey.csv")
student=data.frame(X=5,Y=7)
survey1=survey[,1:2]
pred=knn(survey1,student,survey$Z,k=3)
cat("Predicted Classification of Student:\n")
print(pred)
```

Program 23 - Prediction of Cancer Using k-NN algorithm

```
library(class)
library(gmodels)
wbcd = read.csv("wisc_bc_data.csv")
normalize <- function(x)
{
    return ((x - min(x)) / (max(x) - min(x)))
}
wbcd_n = as.data.frame(lapply(wbcd[3:31], normalize))
wbcd_train = wbcd_n[1:469, ]
wbcd_test = wbcd_n[470:569, ]
wbcd_train_labels = wbcd[1:469, 2]
wbcd_test_labels = wbcd[470:569, 2]
wbcd_test_pred = knn(wbcd_train, wbcd_test, wbcd_train_labels,21)
print(wbcd_test_pred)
CrossTable(x = wbcd_test_labels, y = wbcd_test_pred, prop.chisq=FALSE)
```

Program 24 - Prediction of Flu Using Naive Bayes Algorithm

```
library(e1071)
symptoms = read.csv("symptoms.csv")
symptoms_train = symptoms[,1:4]
symptoms_test = data.frame(Chills="Y",RunningNose="N",Headache="mild",
                           Fever="Y")
classifier_cl <- naiveBayes(symptoms_train,symptoms$HasFlu)
print(classifier_cl)
symptoms_test_pred <- predict(classifier_cl, symptoms_test)
cat("Prediction of Flu:\n")
print(symptoms_test_pred)
```

Program 25 - Prediction of Stealing Using Naive Bayes Algorithm

```
library(e1071)
cars = read.csv("cars.csv")
cars_train = cars[,2:4]
cars_train_labels = cars[,5]
cars_test = data.frame(Colour="red",Type="SUV",Origin="domestic")
classifier_cl <- naiveBayes(cars_train,cars_train_labels)
print(classifier_cl)
cars_test_pred <- predict(classifier_cl, cars_test)
cat("Prediction of Stolen Status:\n")
print(cars_test_pred)
```

Program 26 - Prediction of Iris Species Using Naive Bayes Algorithm

```
library(e1071)
library(caTools)
library(gmodels)
iris = read.csv("iris.csv")
set.seed(100)
split <- sample.split(iris$species, SplitRatio = 0.7)
iris1 <- subset(iris, split == "TRUE")
iris2 <- subset(iris, split == "FALSE")
iris_train = iris1[,1:4]
iris_test = iris2[,1:4]
iris_train_labels = iris1[,5]
iris_test_labels = iris2[,5]
classifier_cl <- naiveBayes(iris_train,iris_train_labels )
print(classifier_cl)
iris_test_pred <- predict(classifier_cl, iris_test)
print(iris_test_pred)
CrossTable(iris_test_labels, iris_test_pred, prop.chisq=FALSE)
```

Program 27 - Prediction of Cheating Using C5.0 Decision Tree Algorithm

```
library(C50)
library(gmodels)
person <- read.csv("person.csv")
person_train <- person[1:8,-5 ]
person_test <- person[9:10,-5 ]
person_train_labels = person[1:8, 5]
person_test_labels = person[9:10, 5]
person_model<-C5.0(person_train,as.factor(person_train_labels))
print(person_model)
print(summary(person_model))
person_pred <- predict(person_model, person_test)
print(person_pred)
CrossTable(person_test_labels, person_pred, prop.chisq=FALSE )
```

Program 28 - Prediction of Cricket Play Using C5.0 Decision Tree Algorithm

```
library(C50)
library(gmodels)
cricket <- read.csv("cricket.csv")
cricket_train <- cricket[1:9,-5 ]
cricket_test <- cricket[10:14,-5 ]
cricket_train_labels = cricket[1:9, 5]
cricket_test_labels = cricket[10:14, 5]
cricket_model<-C5.0(cricket_train,as.factor(cricket_train_labels))
print(cricket_model)
print(summary(cricket_model))
cricket_pred <- predict(cricket_model, cricket_test)
print(cricket_pred)
CrossTable(cricket_test_labels, cricket_pred, prop.chisq=FALSE )
```

Program 29 - Identification of Risky Bank Loans Using C5.0 Decision Tree Algorithm

```
library(C50)
library(gmodels)
credit <- read.csv("credit.csv")
credit_train <- credit[1:900,-17 ]
credit_test <- credit[901:1000,-17 ]
credit_train_labels = credit[1:900, 17]
credit_test_labels = credit[901:1000, 17]
credit_model<-C5.0(credit_train,as.factor(credit_train_labels))
print(credit_model)
print(summary(credit_model))
credit_pred <- predict(credit_model, credit_test)
print(credit_pred)
CrossTable(credit_test_labels, credit_pred, prop.chisq=FALSE )
```

Program 30 - Prediction of Stock Index Price Using Multiple Linear Regression Technique

```
economy <- read.csv("economy.csv")
economy_train <- economy[1:18,]
economy_test <- economy[19:24,]
economy_model <- lm(Stock_Index_Price ~ Interest_Rate + Unemployment_Rate,
                    data = economy_train)
print(economy_model)
print(summary(economy_model))
economy_pred <- predict(economy_model, economy_test)
print(economy_pred)
```

Program 31 - Prediction of Heart Disease Using Multiple Linear Regression Technique

```
heart <- read.csv("heart.csv")
heart_train <- heart[1:400,]
heart_test <- heart[401:498,]
heart_model <- lm(heart.disease ~ biking + smoking, data = heart_train)
print(heart_model)
print(summary(heart_model))
heart_pred <- predict(heart_model, heart_test)
print(heart_pred)
```

Program 32 - Prediction of Medical Expenses Using Multiple Linear Regression Technique

```
insurance <- read.csv("insurance.csv")
insurance_train <- insurance[1:1000,]
insurance_test <- insurance[1001:1338,]
insurance_model <- lm(expenses ~ age + sex + bmi + children + smoker +
                      region, data = insurance_train)
print(insurance_model)
print(summary(insurance_model))
insurance_pred <- predict(insurance_model, insurance_test)
print(insurance_pred)
```

Program 33 - Partition of Iris Data Set Using k-means Clustering Algorithm

```
set.seed(100)
iris <- read.csv("iris.csv")
iris2 <- iris
iris2$species <- NULL
iris_clusters <- kmeans(iris2, 3)
print(iris_clusters)
print(table(iris$species, iris_clusters$cluster))
```

Program 34 - Partition of Breast Cancer Data Set Using k-means Clustering Algorithm

```
set.seed(100)
wisc_bc_data <- read.csv("wisc_bc_data.csv")
wisc_bc_data2 <- wisc_bc_data
wisc_bc_data2$id <- NULL
wisc_bc_data2$diagnosis <- NULL
wisc_bc_data_clusters <- kmeans(wisc_bc_data2, 2)
print(wisc_bc_data_clusters)
print(table(wisc_bc_data$diagnosis, wisc_bc_data_clusters$cluster))
```